API Gateway

什麼是 API Gateway？

想像你的系統從一個單體式應用程式（monolith） 拆成了十個微服務：用戶服務、訂單服務、商品服務、庫存服務、通知服務……每個服務有自己的 IP 和 port，有自己的 API 格式。

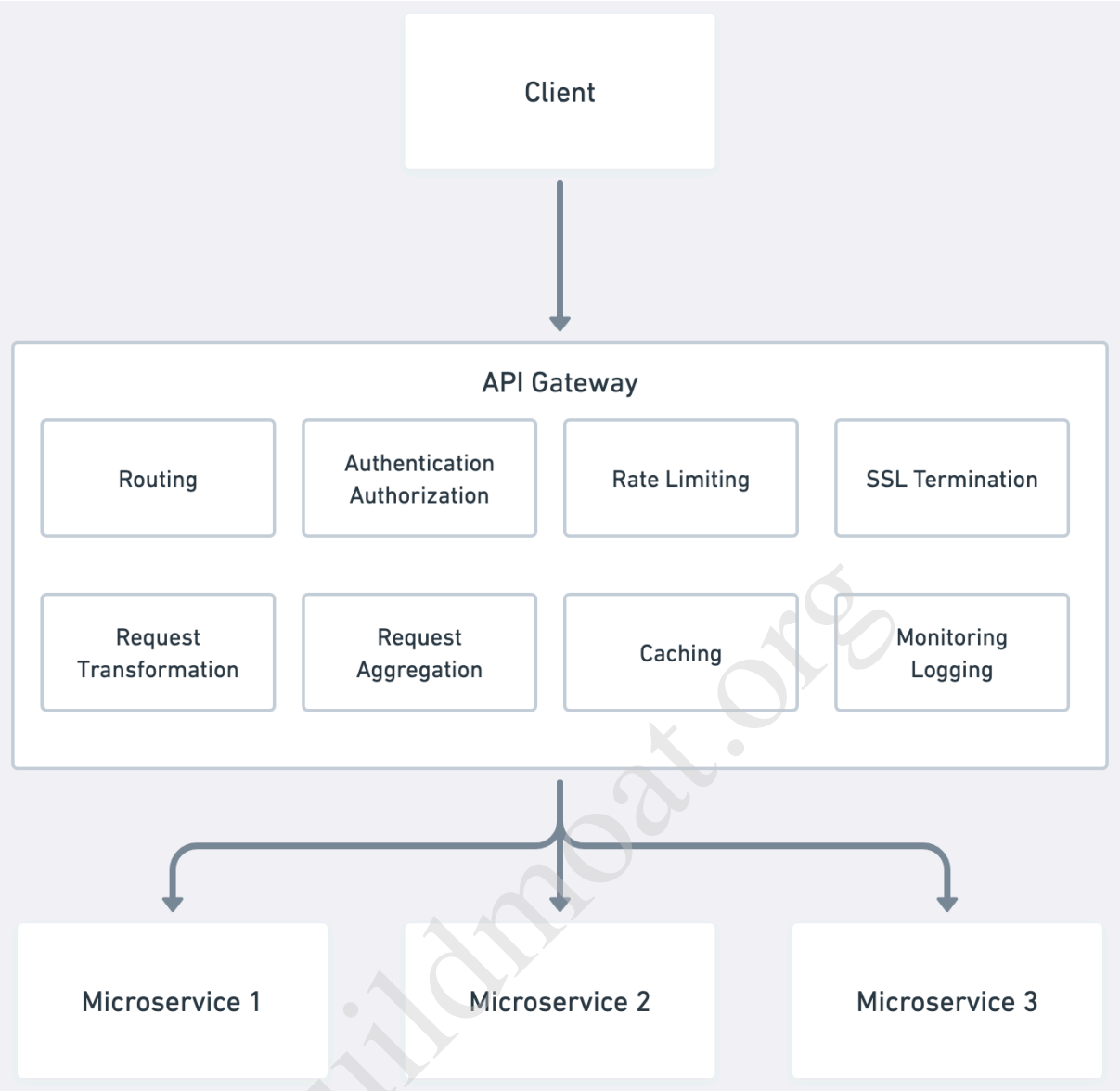
沒有 API Gateway 的情況下，客戶端（手機 App、瀏覽器、第三方合作夥伴）必須直接和每個服務溝通——知道它們各自的地址、各自的協定、各自的認證方式。一個「顯示訂單頁面」的操作可能需要打三個不同的 API：用戶服務取個人資料、訂單服務取訂單列表、商品服務取商品詳情。每個服務都要做 JWT 驗證；每個服務都要做 rate limiting；每個服務都要處理跨來源請求（CORS）。這些橫切關注點（cross-cutting concerns）在每個服務裡重複實作，既浪費又容易出錯。

更麻煩的是，你把內部服務的拓撲（Topology）直接暴露給外部世界。哪天要把訂單服務拆成兩個，或者把某個服務換個 port，所有客戶端的呼叫位址都要更新。

API Gateway 是這個問題的解法。它是客戶端和後端服務之間的統一入口（single entry point）——所有外部請求都先進 API Gateway，由它決定把請求路由到哪個服務，並在中途處理所有橫切關注點。後端服務的細節對客戶端完全透明，客戶端只需要知道一個地址。

在系統設計面試中，API Gateway 幾乎是微服務架構的標配。理解它的責任範圍和取捨，能讓你的設計更完整、更有說服力。

bunenoat.tw



buildmoat.org

API Gateway 的核心職責

請求路由（Routing）

最基本的功能：根據請求的路徑、HTTP method、或 header，把請求轉發到對應的後端服務。

```
GET  /users/{id}        → 用戶服務 (user-service:8001)
GET  /orders/{id}       → 訂單服務 (order-service:8002)
POST /orders            → 訂單服務 (order-service:8002)
GET  /products/{id}     → 商品服務 (product-service:8003)
```

客戶端只看到一個統一的 API，不知道背後有幾個服務、它們跑在哪裡。服務的地址改了、拆分了、合併了，只需要更新 API Gateway 的路由規則，客戶端的呼叫完全不受影響。

認證與授權（Authentication & Authorization）

把認證邏輯集中在 API Gateway，後端服務就不需要各自實作一遍。

常見的流程是：客戶端帶著 JWT token 發請求 → API Gateway 驗證 token 的簽名和有效期 → 驗證通過後，把解析出的用戶資訊（user_id、roles 等）塞進請求 header → 轉發給後端服務。後端服務信任這個 header，不需要再自己解析 JWT。

```
Client                  API Gateway                 後端服務
  |                         |                           |
  |-- GET /orders ------->|                           |
  |   Authorization:        |                           |
  |   Bearer <JWT>          |                           |
  |                         |-- 驗證 JWT 簽名           |
  |                         |-- 解析 user_id： 123      |
  |                         |                           |
  |                         |-- GET /orders -------->|
  |                         |   X-User-Id: 123          |
  |                         |   X-User-Roles: admin     |
  |<-- 200 OK  -----------|<-- 200 OK  ------------|
```

授權可以在 Gateway 做粗粒度的控制——「這個 token 有沒有權限存取 /admin/... 路徑下的 API？」——細粒度的業務邏輯授權（「這個用戶能不能看這筆訂單？」）通常還是在後端服務裡做。

限流（Rate Limiting）

API Gateway 是做限流最自然的地方，因為所有請求都經過它。

限流可以按多個維度：

- **按 API key / 用戶**：免費用戶每分鐘 60 個請求，付費用戶每分鐘 1000 個
- **按 IP**：防止單一來源的大量請求（DDoS 防護的第一道防線）
- **按 endpoint**：敏感操作（登入、發送簡訊驗證碼）的限流比一般 API 更嚴格
- **全域限流**：整個 API 的總請求量上限，保護後端服務不被打垮

buildnoat.org

當請求超過限流閾值，API Gateway 直接回傳 HttpCode （`429 Too Many Requests`），後端服務完全不知道這個請求曾經存在。這保護了後端服務，也讓限流的行為一致可預期。

SSL 終止（SSL Termination）

HTTPS 的加解密（TLS handshake）有 CPU 開銷。讓 API Gateway 處理 SSL，後端服務和 Gateway 之間用 HTTP 在內部網路溝通，這叫做 SSL 終止。

好處是集中管理 TLS 憑證（只需要在 Gateway 更新，不需要各個服務分別管理），以及讓後端服務省去加解密的 CPU 消耗。

在內部網路裡用 HTTP 是否安全，取決於你的網路隔離設計。如果 Gateway 和後端服務在同一個 VPC 內，通常可以接受；如果有更嚴格的安全要求（例如 mTLS），後端服務和 Gateway 之間也可以加密，但那是更複雜的配置。

請求/回應轉換（Request/Response Transformation）

API Gateway 可以在請求到達後端之前、或回應回傳客戶端之前，做格式轉換。

常見的用途：

- **協定轉換**：外部客戶端用 REST，但某個內部服務用 gRPC；Gateway 做 REST 到 gRPC 的轉換
- **欄位過濾**：後端服務回傳的 JSON 包含敏感欄位，Gateway 在回傳客戶端之前把它們過濾掉
- **格式統一**：不同服務的錯誤格式不一致，Gateway 統一轉換成客戶端期待的格式
- **Header 注入**：把用戶資訊、request ID 等 context 注入請求 header，方便後端服務使用

請求聚合（Request Aggregation）

一個客戶端操作需要呼叫多個後端服務，API Gateway 可以把這些呼叫聚合成一個，對外呈現為單一的 API。

```
客戶端請求: GET /dashboard

API Gateway 同時呼叫：
  ├── 用戶服務: GET /users/123                → 姓名、頭像
  ├── 訂單服務: GET /users/123/orders         → 最近訂單
  └── 通知服務: GET /users/123/notifications → 未讀通知
```

buildmoat.org

```
聚合結果後，回傳單一的 JSON 回應
```

這減少了客戶端的網路往返次數，對行動端特別重要，像是手機在 4G 網路下每次 HTTP 請求都有相當的延遲，能少打一個就少打一個。

不過，請求聚合也增加了 Gateway 的複雜度，讓 Gateway 開始「懂業務邏輯」，有些架構師認為 Gateway 應該保持薄薄的一層，不應該做聚合。這是個設計選擇，在面試中說出你的考量就好。

快取（Caching）

對於讀多寫少、回應不常變動的 API，API Gateway 可以快取後端服務的回應，直接在 Gateway 層回傳，後端服務完全不需要處理這些請求。

這對公開的、無需個人化的資料特別有效，比如商品目錄、公告、靜態配置。帶有個人化的回應（「這個用戶的購物車」）通常不適合在 Gateway 快取，因為每個用戶的資料都不同。

API Gateway vs Load Balancer

這是面試中常見的混淆點。兩者都把流量轉發到後端，但工作在不同的層次，解決不同的問題。

Load Balancer 工作在 L4（傳輸層，TCP/UDP）或 L7（應用層，HTTP）。它的工作是把流量**均勻分散**到一組相同的後端實例，以便做健康檢查（Health Check），確保不把請求打到掛掉的機器。它不理解你的業務邏輯，也不管你呼叫的是哪個 endpoint。

API Gateway 工作在 L7，理解 HTTP 請求的語義。它根據請求的**內容**做決策，像是這個路徑要去哪個服務、這個用戶有沒有認證、這個 IP 有沒有超過限流。它是智慧的路由器，而 Load Balancer 是流量分發器。

兩者通常一起使用，而不是互相替代：

```
Internet → Load Balancer → API Gateway 集群 → 各後端服務
                                                ↗ 用戶服務 × 3 實例
                                                ↗ 訂單服務 × 5 實例
                                                ↗ 商品服務 × 2 實例
```

buildmeat.org

Load Balancer 把流量分散到多個 API Gateway 實例（確保 Gateway 本身不成為單點故障）；API Gateway 再根據請求內容路由到對應的後端服務；後端服務前面如果流量夠大，也可以再加 Load Balancer。

	Load Balancer	API Gateway
工作層次	L4 / L7	L7
主要職責	流量分散、高可用	路由、認證、限流、轉換
是否理解業務邏輯	否	是
典型使用場景	同一服務的多個實例	不同服務的統一入口
代表產品	AWS ALB / NLB、Nginx	AWS API Gateway、Kong、Envoy

BFF（Backend for Frontend）模式

微服務架構成熟後，一個常見的演化是：不同的客戶端（Web、iOS、Android、第三方 API）對 API 的需求差異越來越大。

- Web 端需要豐富的資料，因為頻寬和計算能力充足
- 行動端需要精簡的回應，減少流量和解析成本
- 第三方合作夥伴需要穩定的、版本化的 API，不能因為內部系統的變動而改變

用同一個 API Gateway 服務所有客戶端，你會發現自己在不斷為不同客戶端做各種妥協。BFF（Backend for Frontend）模式的解法是：**為不同類型的客戶端維護各自的 Gateway**。

```
Web Browser    → Web BFF      ─┐
iOS App        → Mobile BFF   ─┼─→ 後端微服務
Android App    → Mobile BFF   ─┘
Third-party    → Public API Gateway
```

Web BFF 可以做大量的資料聚合，回傳豐富的 JSON；Mobile BFF 回傳精簡的欄位；Public API Gateway 維護嚴格的版本控制和向後兼容。每個 BFF 由對應的前端團隊擁有和維護，後端微服務保持不變。

代價是你現在有多個 Gateway 要維護。BFF 適合有多個差異明顯的客戶端、且每個客戶端團隊有能力維護自己的 BFF 的場景；對於小型團隊或差異不大的客戶端，一個統一的 API Gateway 就夠了。

常見的 API Gateway 實作

buildmoat.org

不同場景有不同的選擇，在面試中說出你選擇的理由比說出正確名字更重要。

AWS API Gateway 是全受管的 Serverless Gateway，和 Lambda 的整合極為緊密。你不需要管理任何伺服器，它會自動擴展（auto scaling），按請求計費。適合 Serverless 架構、或者不想自己管理 Gateway 基礎設施的場景。缺點是進階的路由規則和自訂邏輯受到一些限制，成本在高流量下也比自架高。

Kong 是開源的 API Gateway，基於 Nginx 構建，透過插件系統擴展功能（認證、限流、logging 等）。可以自架也可以用 Kong Cloud。適合需要高度客製化、或已有 Nginx 運維經驗的團隊。

Envoy / Istio 是服務網格（Service Mesh）的核心。在 Kubernetes 環境裡，Envoy 作為 sidecar proxy 部署在每個 Pod 旁邊，處理服務間的流量（mTLS、熔斷、追蹤）。API Gateway 的功能（外部流量入口）和服務網格的功能（內部服務間流量）有所重疊，但通常是分開的，比如Ingress Gateway 處理外部流量，Service Mesh 處理內部流量。

Nginx / Traefik 是比較輕量的選擇。Nginx 幾乎人人熟悉，設定靈活；Traefik 對 Kubernetes 的支援很好，能自動發現服務和更新路由規則。適合不需要複雜功能、只要基本路由和 SSL 終止的場景。

什麼時候在面試裡用這些

提到微服務時，主動說明 API Gateway

任何微服務架構的設計，都應該明確說明對外的入口。

「這個系統拆成了用戶服務、訂單服務、商品服務。對外有一個 API Gateway，負責路由、JWT 認證、和限流。客戶端只需要知道 Gateway 的地址，後端服務的拆分和部署對客戶端透明。」

說不說 API Gateway，是面試中體現架構成熟度的細節之一。

討論安全性時提認證集中化

「認證邏輯集中在 API Gateway：Gateway 驗證 JWT、解析用戶資訊，透過 header（`X-User-Id`、`X-User-Roles`）傳給後端服務。後端服務信任這個 header，不需要各自實作認證邏輯。細粒度的業務授權（這個用戶能不能存取這筆訂單）還是在服務層做。」

討論系統保護時提限流

「API Gateway 做限流的第一道防線，按 IP 和按 API key 限制請求頻率，防止流量突增打垮後端服務。超過限制的請求直接在 Gateway 層回 429，後端服務完全不感知。」

buildmoat.org

常見面試情境

電商平台（高流量促銷活動）：「雙十一期間，Gateway 的限流規則是關鍵，像是對一般用戶每秒限 10 個請求，防止搶購 Bot 刷爆系統。同時，Gateway 快取商品目錄 API（60 秒 TTL），商品詳情頁的流量大部分在 Gateway 就消化掉，不打到商品服務。」

多平台 App（Web + Mobile）：「Web 端和行動端的資料需求差很多，Web 端的訂單頁面需要訂單、商品詳情、物流狀態一次全拿；行動端因為流量限制只需要訂單列表和基本狀態。我們用 BFF 模式，分別維護 Web BFF 和 Mobile BFF，各自根據客戶端需求聚合後端服務的資料。」

開放 API 平台（提供給第三方）：「第三方合作夥伴透過 Public API Gateway 存取，和內部系統的 API 分開。Public Gateway 做 API key 認證、嚴格的限流、以及版本管理（`/v1/`、`/v2/`）。版本之間的兼容性由 Gateway 維護，後端服務不需要處理多版本的相容問題。」

常見的 Deep Dive 問題

「API Gateway 本身是不是單點故障？」

這是面試官測試你是否考慮了高可用性的問題。

答案是：Gateway 本身必須多實例部署，前面掛 Load Balancer。

```
Internet → AWS ALB / NLB → API Gateway 實例 × N
```

多個 Gateway 實例同時運行，Load Balancer 負責分散流量和健康檢查。某個 Gateway 實例掛掉，Load Balancer 把它移出輪換，不影響整體服務。

因為 Gateway 通常是**無狀態**的（路由規則存在設定檔或資料庫，限流狀態存在 Redis），任何一個實例都能處理任何請求，水平擴展非常自然。

限流的狀態需要跨實例共享——如果 A 實例看到用戶發了 50 個請求，B 實例不知道這件事，用戶就能繞過限制。解法是把限流的計數器存在共享的 Redis 或 Memcached：

```
API Gateway 實例 A ─┐
API Gateway 實例 B ─┼─→ Redis (限流計數器)
API Gateway 實例 C ─┘
```

「怎麼做認證：在 Gateway 還是在服務裡？」

這不是一個非此即彼的問題，正確答案是兩個層次都做，但做不同的事。

buildnoat.org

Gateway 做認證（Authentication）：驗證 JWT 的簽名和有效期，確認「這個 token 是真的、還沒過期」。這是技術層面的驗證，和業務邏輯無關，非常適合在 Gateway 集中處理。

服務做授權（Authorization）：驗證「這個用戶能不能做這件事」。這和業務邏輯緊密相連——「用戶只能看自己的訂單」這個規則需要資料庫查詢，Gateway 沒有這個上下文，必須在服務層做。

Gateway 可以做粗粒度的授權：「這個路徑需要 admin role，而這個 token 的 roles 欄位沒有 admin，拒絕」。但細粒度的資料層授權（這個資源屬不屬於這個用戶？）要留在服務裡。

一句話總結：**Gateway 保證你是你說的那個人，服務決定你能做什麼。**

「API Gateway 的效能怎麼保證？」

API Gateway 在請求路徑上增加了一個中間層，必然有延遲。常見的問題是：這個開銷有多大？怎麼最小化？

路由和認證的開銷：路由規則查詢通常在記憶體裡完成，幾乎是零延遲。JWT 驗證是本地密碼學運算，不需要網路請求，也非常快。合理實作下，Gateway 帶來的額外延遲通常在 1 到 5 毫秒以內。

外部認證服務：如果 Gateway 需要呼叫外部服務做認證（例如呼叫 Auth0、或呼叫自己的 User Service 驗證 session），每個請求都多一次網路往返，延遲可能增加幾十毫秒。解法是在 Gateway 快取認證結果（例如把驗證通過的 JWT 快取 60 秒），或者盡量用無需外部呼叫的 JWT（自帶簽名、本地驗證）。

連線池：Gateway 和後端服務之間要維護連線池，避免每個請求都重新建立 TCP 連線。合理的連線池配置對效能影響很大，特別是 HTTPS 連線，TLS handshake 的成本不低。

水平擴展：Gateway 是無狀態的，流量增加時直接加實例。在容量規劃上，Gateway 的 CPU 使用率主要來自 TLS 終止和 JWT 驗證，這兩者都是 CPU bound；如果 Gateway 成為瓶頸，加實例或換更強的機器都能解決。

「怎麼處理 Gateway 的設定更新？」

API Gateway 的路由規則、限流設定、認證規則——這些設定需要更新，而更新不能影響正在處理的請求。

設定熱重載（Hot Reload）：更新設定後，Gateway 不需要重啟，直接載入新設定。大多數 API Gateway 支援這個，Kong 和 Nginx 的設定變更可以不停機生效。

金絲雀部署（Canary Routing）：新版本的路由規則先只對 5% 的流量生效，觀察沒有問題後再全量推。API Gateway 是做金絲雀部署的好地方——直接在 Gateway 層控

buildpoat.org

制流量分配比例，不需要動後端服務。

```
GET /orders → 95% → 訂單服務 v1.0
            → 5%  → 訂單服務 v2.0（金絲雀）
```

版本管理：路徑前綴 `/v1/` 、 `/v2/` 是最常見的 API 版本管理方式。Gateway 根據版本前綴路由到不同的後端（或同一個後端的不同版本），讓 v1 和 v2 可以共存，舊客戶端不需要立刻升級。

總結

API Gateway 是微服務架構的「前門」——它把客戶端和後端服務解耦，把橫切關注點集中管理，讓後端服務專注在業務邏輯上。

在系統設計面試中，提到微服務就應該說明 API Gateway 的角色。不只是「有個 Gateway 做路由」，而是說清楚它承擔了哪些職責（認證、限流、SSL 終止）、為什麼這些東西集中在 Gateway 比分散在每個服務裡更好（統一管理、避免重複、後端服務解耦）。

記住幾個容易被問到的點：**Gateway 本身需要多實例部署避免單點故障**；**認證在 Gateway**、**授權在服務**；**限流計數器需要共享 Redis**。這三個細節說出來，展示你不只是說了「用 API Gateway」，而是真的想清楚了它在分散式系統裡的定位。

API Gateway 不是萬能的：把太多業務邏輯塞進 Gateway，它會變成一個難以測試和維護的大泥球；請求聚合的功能讓 Gateway 開始「懂業務」，要謹慎使用。Gateway 的角色應該是一個薄薄的、可預期的轉發層，而不是第二個 monolith。

buildmost.org